

LANBOX

An Integrated Local Area Network Communication and Collaboration Platform

PROJECT PROPOSAL / TECHNICAL DOCUMENT

Team Members

Kautham Kanthavel P J

Pranesh S

Domain: Computer Networks

Project Type: LAN Communication, Collaboration and Remote Assistance Platform

Table of Contents

1. Project Overview
2. Problem Statement
3. Proposed Solution
4. Objectives
5. Scope of the Project
6. Major Features
7. System Architecture
8. How the System Works
9. Technology and Protocol Selection
10. Module-wise Implementation Approach
11. Communication Flow
12. Remote Desktop and Screen Sharing Approach
13. Security and Permission Model
14. Database and Data Management
15. User Interface
16. Development Roadmap
17. Testing and Demonstration Plan
18. Expected Outcome
19. Future Enhancements
20. Conclusion
21. Short Project Abstract

1. Project Overview

LANBOX is a LAN-based communication and collaboration platform intended for users connected to the same local area network. The platform combines messaging, file sharing, user discovery, voice and video communication, screen sharing, and controlled remote assistance within one application. The project is designed as a practical demonstration of computer networking concepts rather than as a simple chat application.

The core idea is to provide a controlled local environment in which computers can discover one another, establish communication sessions, exchange data, share media, and request collaborative or remote-support operations. A central LANBOX service will coordinate users, authentication, signalling, sessions, and permissions, while appropriate transport mechanisms will be used for different types of traffic.

2. Problem Statement

In classrooms, computer laboratories, small offices, and institutional LANs, users often depend on multiple independent applications for chat, file transfer, voice/video meetings, screen sharing, and remote assistance. This creates fragmented workflows and can introduce unnecessary dependence on external services when the participants are already connected to the same local network.

The problem addressed by LANBOX is therefore the absence of a unified, LAN-oriented platform that can provide reliable data communication, real-time media communication, resource sharing, and controlled remote assistance through one system while demonstrating the underlying networking concepts.

3. Proposed Solution

LANBOX proposes a modular client-server system. A LANBOX server manages connected users and coordinates authentication, messaging, discovery, file-transfer requests, call signalling, collaboration sessions, and access permissions. Client applications provide the user interface and interact with the server and, where appropriate, with another client for real-time media.

The design intentionally uses different communication mechanisms according to the requirement. Reliable TCP-based communication is suitable for messages, authentication, control commands, and file transfer. UDP can be used for LAN discovery and other latency-sensitive mechanisms where appropriate. For voice, video, and screen sharing, WebRTC-style real-time communication is proposed because it provides APIs for media capture and peer connections. WebRTC supports audio, video and data communication, and its media APIs include camera, microphone and display capture. These choices make the project technically meaningful and easier to explain during a networking viva.

4. Objectives

- Build a practical LAN communication platform using socket and network programming concepts.
- Allow users on the same LAN to discover and communicate with each other.
- Implement one-to-one and group messaging.
- Provide reliable LAN file transfer.
- Provide voice and video calling capabilities.
- Enable user-controlled screen sharing.
- Provide permission-based remote desktop assistance with mouse and keyboard operations.
- Demonstrate TCP, UDP, IP addressing, ports, sockets, concurrent connections, streaming, and session management.

- Provide authentication and explicit permission controls for sensitive operations.
- Maintain a modular architecture that can be tested and extended.

5. Scope of the Project

The initial scope is a controlled local network such as a college laboratory, classroom, office, or home LAN. The first implementation will prioritize communication between computers on the same network. Internet-wide communication is not the primary goal.

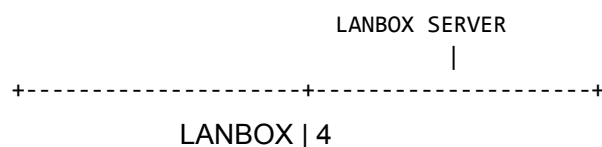
The project will be developed incrementally. Basic socket communication and messaging will be completed first, followed by file transfer and LAN discovery. Real-time voice/video and screen sharing will then be integrated. Remote assistance will be implemented only with explicit authorization from the receiving computer.

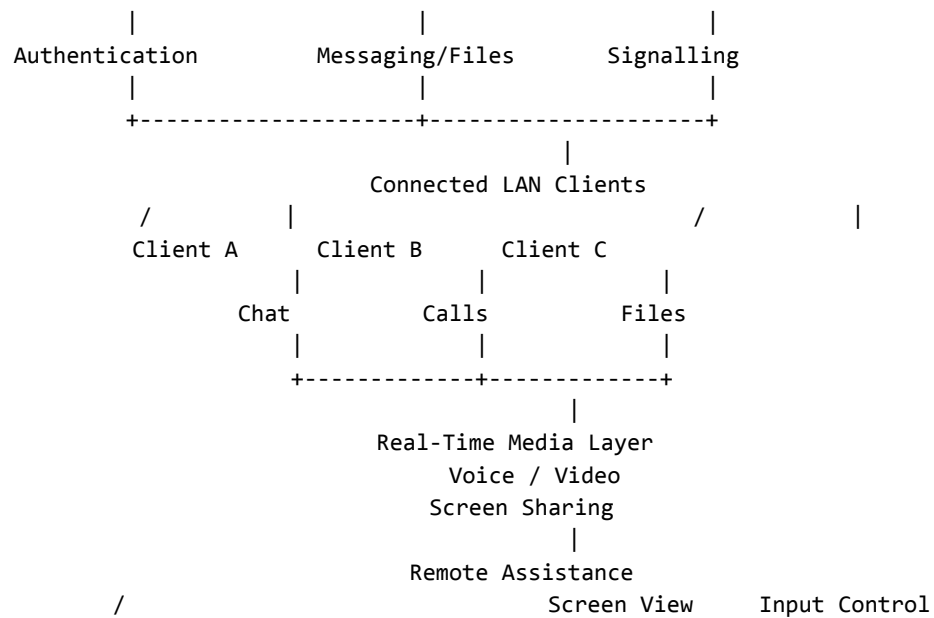
6. Major Features

Feature	Purpose
User Authentication	Login, identity management, session creation and logout.
LAN User Discovery	Identify active LANBOX clients and maintain online/offline status.
Private Messaging	One-to-one text communication with timestamps and message history.
Group Messaging	Create groups and distribute messages to selected members.
File Sharing	Send files over the LAN with metadata and transfer status.
Voice Calling	Establish real-time audio sessions between authorized users.
Video Calling	Transmit camera video and audio during a call.
Screen Sharing	Share a selected display/window with another authorized user.
Remote Assistance	Permission-based remote screen viewing and controlled mouse/keyboard input.
Announcements	Send network-wide or group-specific notices.
Session Management	Track active chats, calls, file transfers and remote-support sessions.
Network Dashboard	Display server IP, port, connected users and active sessions.
Security Controls	Authentication, permission prompts, session termination and access restrictions.

7. System Architecture

The proposed architecture separates control traffic from high-volume real-time media wherever practical. The server acts as the central control point for authentication, user presence, messaging, file-transfer coordination, signalling, and permission management. Clients handle media capture, display, and approved remote-control operations.





The architecture is modular so that a failure or change in one feature does not require rewriting the entire application. For example, the messaging subsystem can use TCP sockets while the real-time calling subsystem can use WebRTC-style peer connections.

8. How the System Works

1. Server startup: The LANBOX server starts on a computer connected to the LAN and listens on configured ports.
2. Client startup: A LANBOX client connects to the server using the server's local IP address and port.
3. Authentication: The client submits credentials or a registered identity. The server validates the request and creates a session.
4. Presence: The server records the user's active session and distributes presence information to permitted clients.
5. Communication: Chat messages are sent through the messaging service and delivered to the intended user or group.
6. File sharing: File metadata is exchanged and the file is transferred through a reliable channel, with progress information maintained.
7. Calling: The server coordinates call setup/signalling. The clients establish the real-time media connection.
8. Screen sharing: The sender captures the permitted screen/window and sends it as a real-time media stream.
9. Remote assistance: The controller requests access; the remote user explicitly accepts. Only then are authorized screen/input operations enabled.
10. Termination: Users can end calls, sharing, transfers or remote-control sessions, and the server removes inactive sessions.

9. Technology and Protocol Selection

Technology	Role
Python	Primary implementation language for networking, server logic and rapid prototyping.
Socket Programming	Low-level network communication using IP addresses, ports, TCP and UDP.
TCP	Reliable, ordered communication for chat, authentication, control messages and file transfer.
UDP	Lightweight datagrams for LAN discovery/broadcast and selected low-latency tasks.
WebRTC	Proposed real-time layer for audio, video and screen sharing. WebRTC provides media capture and RTCPeerConnection APIs for peer communication.
PyQt/PySide	Desktop graphical user interface for the LANBOX client.
SQLite	Local database for users, messages, file metadata and session history.
JSON	Structured control and signalling messages.
Screen Capture Library	Captures permitted desktop/window frames for screen sharing.
OS Input APIs	Generates permitted mouse/keyboard events during an authorized remote-support session.

Python's socket module provides access to the operating system's socket interface and is available on Windows and other major platforms. Qt also exposes TCP and UDP socket abstractions. For real-time media, WebRTC documentation describes camera/microphone capture, display capture, peer connections, and real-time audio/video/data communication.

穀cite學turn0search14學turn0search10學turn0search0學turn0search1係

10. Module-wise Implementation Approach

Module 1: TCP Server and Client: Create socket, bind server IP/port, listen, accept clients, send and receive data, handle disconnects.

Module 2: Multiple Client Handling: Use threads or asynchronous I/O so multiple clients can communicate simultaneously.

Module 3: Authentication: Implement registration/login, password hashing, sessions and logout.

Module 4: Messaging: Define message format, private chat, group chat, timestamps and message history.

Module 5: LAN Discovery: Use UDP broadcast or another LAN discovery mechanism to locate LANBOX services and maintain presence.

Module 6: File Transfer: Transfer files in chunks over a reliable channel and show progress/status.

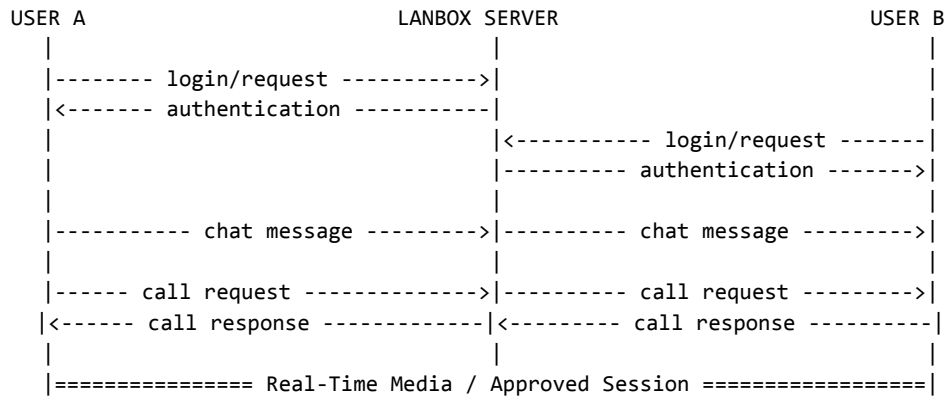
Module 7: Voice/Video: Integrate real-time media capture and peer connections; use signalling through the LANBOX control layer.

Module 8: Screen Sharing: Capture a user-approved screen/window and transmit it as a real-time video stream.

Module 9: Remote Assistance: Implement request/accept/reject, remote screen viewing and authorized mouse/keyboard event transmission.

Module 10: GUI and Integration: Combine modules into a unified desktop interface and test all workflows.

11. Communication Flow



12. Remote Desktop and Screen Sharing Approach

Screen sharing and remote desktop control will be treated as two related but separate capabilities. Screen sharing means transmitting a user's display as a media stream. Remote control additionally transmits input events such as mouse movement, clicks and keyboard actions.

The proposed workflow is: (1) user selects Share Screen or Request Remote Assistance, (2) LANBOX identifies the target user, (3) a permission request is shown on the target computer, (4) the target user accepts or rejects, (5) only an accepted session is established, (6) the target can terminate the session at any time. This design prevents silent remote access.

WebRTC's display-media APIs support screen capture and treat the captured display as a media stream, making the technology suitable for the screen-sharing part of the design. 穀cite學turn0search2學turn0search6像

13. Security and Permission Model

- Authentication before access to LANBOX services.
- Password hashing rather than storing plaintext passwords.
- Session identifiers to prevent unauthorized control requests.
- Permission prompts before screen sharing.
- Separate permission for remote keyboard/mouse control.
- Ability for the controlled user to terminate a remote session immediately.
- Server-side validation of client requests.
- File-transfer validation and configurable size/type restrictions.
- Logging of important security-sensitive session events.
- Optional TLS/encrypted control channels as an enhancement.

14. Database and Data Management

SQLite can be used initially because it is lightweight and suitable for a prototype. The following logical tables are proposed:

Table	Important fields
Users	user_id, username, password_hash, created_at
Sessions	session_id, user_id, login_time, logout_time, status
Messages	message_id, sender_id, receiver/group_id, message, timestamp
Groups	group_id, group_name, created_by
GroupMembers	group_id, user_id
Files	file_id, sender_id, receiver_id, filename, size, timestamp, status
Calls	call_id, caller, receiver/group, type, start_time, end_time
RemoteSessions	session_id, requester, target, permission, start_time, end_time

15. User Interface

The client application will contain a dashboard with the user's profile, online LANBOX users, chat area, file-transfer controls, call controls, screen-sharing controls, remote-assistance controls and network status. The interface will hide low-level socket operations from the end user while the underlying implementation remains available for demonstration and viva explanation.

+-----+ LANBOX User: Kautham LAN: Connected +-----+	
ONLINE USERS	CHAT / CALL / COLLABORATION
* Pranesh	Messages
* User 2	[message box] [Send]
* User 3	
	[Voice] [Video] [Share Screen]
	[Send File] [Remote Assist]
+-----+	
Network: Connected Users: 5 Server: 192.168.x.x +-----+	

16. Development Roadmap

Phase	Work
Phase 1	Learn and implement TCP client-server communication.
Phase 2	Support multiple clients and concurrent connections.
Phase 3	Implement authentication and user sessions.
Phase 4	Build private/group messaging.
Phase 5	Implement LAN discovery and presence.
Phase 6	Implement reliable file transfer.
Phase 7	Create the GUI and integrate the communication modules.

Phase 8	Integrate voice/video communication and signalling.
Phase 9	Add screen sharing.
Phase 10	Add permission-based remote assistance.
Phase 11	Add database, logs, security hardening and testing.
Phase 12	Final LAN demonstration, documentation and viva preparation.

17. Testing and Demonstration Plan

- Run the server on one LAN computer and connect two or more clients.
- Verify login and logout with multiple users.
- Send private and group messages.
- Transfer small and large files and verify file integrity.
- Disconnect/reconnect a client and verify presence handling.
- Initiate and terminate voice/video calls.
- Start and stop screen sharing.
- Attempt remote assistance and verify that access is impossible without acceptance.
- Terminate an active remote session from the controlled computer.
- Measure basic latency, transfer time and behaviour under multiple clients.
- Test invalid requests and unauthorized operations.

18. Expected Outcome

The expected outcome is a working LANBOX prototype in which multiple computers connected to the same LAN can communicate and collaborate through a single platform. The system should demonstrate text communication, file sharing, user discovery, real-time voice/video communication, screen sharing, and controlled remote assistance. In addition to the application itself, the project will demonstrate practical understanding of TCP/IP, UDP, sockets, ports, concurrency, network discovery, real-time media, signalling, and access control.

19. Future Enhancements

- Multi-user video conferences.
- End-to-end encryption and stronger certificate-based authentication.
- Adaptive media quality based on LAN bandwidth.
- Offline message queueing.
- Advanced file synchronization.
- Role-based administration for laboratories and offices.
- Remote session recording with explicit consent.
- Cross-platform client applications.
- Network performance analytics.
- Optional operation across routed networks with appropriate security and NAT traversal.

20. Conclusion

LANBOX is proposed as a unified LAN communication and collaboration platform that brings together several real-world networking functions in a single system. Instead of treating networking as an isolated socket-programming exercise, the project applies networking concepts to practical services such as

messaging, file transfer, calling, screen sharing and remote assistance. The modular development approach allows the system to begin with fundamental TCP/UDP socket programming and gradually introduce real-time communication, media streaming, permissions, security and GUI integration.

21. Short Project Abstract

LANBOX is an integrated Local Area Network communication and collaboration platform designed to provide messaging, file sharing, user discovery, voice calling, video calling, screen sharing and permission-based remote assistance among users connected to the same LAN. The proposed system uses a client-server architecture with socket programming for reliable control and data communication, TCP for connection-oriented services such as messaging and file transfer, UDP for suitable LAN discovery and low-latency tasks, and a WebRTC-based real-time communication layer for audio, video and screen sharing. A graphical client interface will provide access to the major services while a central server manages authentication, user presence, signalling, sessions and permissions. Remote desktop functionality will require explicit authorization before screen viewing or mouse/keyboard control is enabled. The project demonstrates practical computer networking concepts including IP addressing, ports, sockets, TCP/UDP communication, concurrent connections, network discovery, data streaming, session management and access control. The final system is intended for controlled environments such as classrooms, laboratories and offices and will provide a unified platform for LAN-based communication and collaboration.

References Used for Technical Design

- Python Documentation — socket module: low-level networking interface.
- Qt for Python Documentation — TCP/UDP socket abstractions.
- WebRTC Official Documentation — real-time audio, video and data communication.
- WebRTC Getting Started Documentation — media capture, display capture and peer connections.
- W3C WebRTC Recommendation — WebRTC API and real-time peer communication.